Website Security Review

Customer portal of Acme Corp (a fictional company)

PREPARED BY
Farhan Hussain, Penetration Tester (CPENT)

REPORT DATE
3 November 2026

WHAT WAS REVIEWED
The Acme customer portal and the server behind it

HOW IT WAS REVIEWED
Authorised, controlled attack simulation. Nothing was damaged.

> **About this sample.** The company, the system and every detail here are made up for demonstration, built from lab practice. It shows how I present findings so that the people who decide and pay for the fixes can read them without a technical background. It is a short excerpt: a full engagement report adds screenshots and complete evidence for every finding. The matching **Technical Appendix** (separate PDF) shows that level of detail for the IT team.

1. Summary for leadership

WHAT WE DID

We acted like a determined outsider and tried to break into the customer portal using only what anyone on the internet can reach. We had written permission and agreed limits.

WHAT WE FOUND

An outsider with basic skills could take full control of the system in under an hour. This took five separate weaknesses, each small on its own, used one after another.

WHAT IT MEANS FOR THE BUSINESS

Customer records and staff login details could be copied, and the whole system could be taken over. That puts customer privacy, legal obligations and the company's reputation at risk.

WHAT WE RECOMMEND

Fix the two most serious problems this week. All five can be fixed within two months, and most of the work is small. A short re-check afterwards will confirm everything is closed.

Overall risk: High. Act this week.

1	2	1	1
CRITICAL	HIGH	MEDIUM	LOW

Critical: fix immediately. **High:** fix within a week. **Medium:** fix within a month. **Low:** fix when convenient.

2. What we looked at, and how

1. **Look around.** Find out what is visible from the internet: pages, services and anything left lying around.
2. **Try to get in.** Test whether the weaknesses we spotted can really be used, using the smallest safe step.
3. **See how far it goes.** Check what an attacker could reach once inside: data, accounts, control of the server.
4. **Write it up.** Explain what we found, how we know, why it matters and exactly how to fix it.

Not tested: anything that could take the site offline, tricking staff, and services run by other companies.

3. Findings at a glance

NO.	WHAT WE FOUND	RISK	FIX WITHIN
1	The report export page can be tricked into handing over private data	CRITICAL	Immediately
2	A staff note containing the administrator password sits on the public website	HIGH	1 week
3	One staff account has far more power on the server than it needs	HIGH	1 week
4	Passwords are stored in a form that is easy to unscramble	MEDIUM	1 month
5	The system reveals more about itself than it should	LOW	2 months

4. How the weaknesses fit together

1. A note left on the public website revealed the administrator login.
2. We logged in as the administrator, a normal-looking visit that raised no alarm.
3. A flaw in the export page let us copy the list of staff accounts and their stored passwords.
4. One stored password was easy to unscramble, giving us a real staff login.
5. That staff account had extra power on the server, which let us become the top administrator.

5. The most serious finding

1. The report export page can be tricked into handing over private data CRITICAL

What we found. The export page asks the database for one record at a time. It does not check what it is asked, so a specially written request makes it return every record instead of one.

Why it matters. Anyone on the internet can copy the full list of accounts and stored passwords without logging in properly. From there an attacker can impersonate staff or customers.

How we know. We changed the number at the end of the export page address to include one extra instruction. The page replied with every account in the system, not just the first.

> **Result seen:** a table listing all accounts, including the administrator and a deployment account, each with its stored (scrambled) password. Sensitive values are hidden in this sample.

What to do. Have the development team change how the page talks to the database so that input is never treated as an instruction. This is a standard, well-understood change. Also limit what the website's database account is allowed to read, so a future mistake cannot expose everything. **Effort:** a few days of developer time.

6. The other findings

2. A staff note with the administrator password is public HIGH

What we found. A file left on the website says where the admin area is and gives its username and password. A comment in the home page points straight to it.

Why it matters. It is the easiest way in: no skill needed, only reading. **What to do.** Remove the file, change the password now, and add a pre-launch check that looks for forgotten notes and secrets. **Effort:** under an hour, plus the check.

3. One staff account has too much power HIGH

What we found. A deployment account may run a general-purpose file tool as the top administrator without a password. That tool can also run any other command, so the limit does not hold.

Why it matters. Taking over this one account gives an attacker the whole server. **What to do.** Remove that permission, or allow only the single task the account really needs. **Effort:** about an hour.

4. Passwords are easy to unscramble MEDIUM

What we found. Passwords are stored using an old method that modern computers can reverse for common passwords in a fraction of a second. One staff password was recovered this way.

What to do. Move to a current method designed to be slow to unscramble, then ask everyone to choose a new password. **Effort:** a few days, planned with a password reset.

5. The system reveals too much about itself LOW

What we found. Error pages and server messages show software names and versions. **Why it matters.** It helps an attacker choose the right tool. **What to do.** Show plain error pages and hide version details. **Effort:** under a day.

7. Plan and timing

WHEN	WHAT	WHO
This week	Fix finding 1 and remove the public note (2). Change the admin password. Remove the extra server permission (3).	Development and IT
Within 30 days	Upgrade password storage and reset passwords (4).	Development
Within 60 days	Tidy error pages (5). Add checks before each release. Re-test to confirm all five are closed.	IT, with the tester

8. Closing view

Nothing here needed advanced attack skills, which is exactly why it matters: ordinary mistakes lined up. The fixes are clear and mostly inexpensive. We recommend a short re-test once they are done so the company can show customers and partners that the issues are closed.

For your IT team (reference). 1: SQL injection in /export?id, OWASP A03:2021, CWE-89, CVSS 9.8. 2: Credentials in a public file, CWE-538, CVSS 7.5. 3: sudo NOPASSWD on /usr/bin/find, CWE-269, CVSS 7.8. 4: Unsalted MD5 hashes; use Argon2id or bcrypt. 5: Verbose banners and error messages, CVSS 3.7. Scoring: CVSS v3.1. Method: black-box test aligned with PTES and the OWASP Top 10. Full evidence and reproduction steps are in the separate Technical Appendix. This sample is fictional.